

Ingeniería y Calidad de Software

Notas unificadas - Clase del 25/08/2026

1. Requerimientos ágiles y foco de evaluación

Temas señalados
User Story / Historia de Usuario como técnica para materializar requerimientos ágiles.
Para el primer parcial aparecen señalados User Story y MVP.
El apunte complementario remarca la creación de User Stories completas: frase verbal, sintaxis de Card, criterios de aceptación, pruebas de usuario, estimación y definición de MVP.

2. SCM vs. herramientas de versionado

Versión e ítem de configuración
Una versión identifica unívocamente un ítem de configuración y permite controlar sus cambios.
SCM no equivale a usar una herramienta de versionado. Git maneja versiones internas (hashes), pero el nombre de una versión para un release o línea base lo define el equipo.
No usar el hash o número interno de Git como nombre del ítem. El versionado semántico puede responder a convenciones del equipo o políticas del Plan de Gestión de Configuración.

3. Filosofía Ágil, empirismo y Manifiesto

Filosofía Ágil	
Ágil no es una metodología.	
Surge desde desarrolladores que buscaban otra forma de trabajar frente al descarte de productos/proyectos.	
Manifiesto Ágil: 2001.	
El foco no está solamente en el cómo, sino también en el qué.	
Proceso	Características / ejemplos
Definidos	Predictivos. En las notas aparecen PUD y RUP.
Empíricos	Agile y Lean: prueba, error y realimentación.
Pilares del empirismo	
Inspección.	
Adaptación.	
Transparencia.	

4 valores del Manifiesto Ágil

Valor	Interpretación
Individuos e interacciones	Sobre procesos y herramientas. El equipo auto-organizado puede adaptar su forma de trabajo.
Software funcionando	Sobre documentación exhaustiva/detallada. El valor real está en el software funcionando; no se prohíbe documentar.
Colaboración con el cliente	Sobre negociación contractual. El cliente o PO participa estrechamente con el equipo.
Respuesta ante el cambio	Sobre seguir un plan. Se acepta que los requerimientos cambiarán.

4. Principios ágiles destacados

Trabajo conjunto y auto-organización
Responsables de negocio, diseñadores y desarrolladores deben trabajar juntos día a día.
El método más eficiente de comunicar información es la conversación cara a cara.
Las mejores arquitecturas, requisitos y diseños emergen de equipos auto-organizados.
Quienes tienen que tomar decisiones son los integrantes del equipo.

5. Product Owner: rol del negocio

Responsabilidad
Debe ser alguien del negocio, con conocimiento, presupuesto y autoridad para tomar decisiones.
Tiene Accountability sobre el QUÉ: producto y priorización. El equipo técnico decide el CÓMO: arquitectura y tecnologías.
El Product Owner ideal es uno solo. Si no puede participar, en las notas aparece un proxy que debe saber y poder tomar decisiones.
Output vs. Outcome
No alcanza con entregar líneas de código que compilen (output); se busca entregar valor, beneficios y resultados para el usuario (outcome).
El apunte complementario indica que el 50 % de los requerimientos son emergentes y menciona Pareto: 80 % del valor del producto reside en 20 % de las funcionalidades.

6. Requerimientos ágiles y valor

Ideas centrales
No es necesario ni posible tener el 100 % de los requerimientos antes de empezar.
Usar el valor para construir el producto correcto.
No es valor entregar código que compile; se vuelve valor cuando satisfacemos al cliente.
Los requerimientos son emergentes: se desarrolla una versión para descubrir nuevos requerimientos.

7. Product Backlog y granularidad

Product Backlog	
Lista priorizada por valor de negocio y responsabilidad del Product Owner.	
Nunca está completo y no equivale a una ERS.	
Trabaja bajo Just-In-Time: especificar en el último momento responsable.	
Nivel	Descripción
Tema	Gran agrupador de ideas.
Épica	Historia demasiado grande para entrar en un solo Sprint; debe dividirse.
User Story	Unidad suficientemente pequeña para consumirse en una iteración. Se corta verticalmente para entregar valor.
Corte vertical	
Presentación/UI + lógica/servicio + persistencia/base de datos.	
Prioridad y Just in Time	
Prioridad alta: lo que se tiene que hacer sí o sí.	
Analizar cuando se necesita, no antes; no invertir tiempo anticipadamente en todos los requerimientos.	

8. User Stories: definición, 3C y sintaxis

Definición	
Descripción corta de una necesidad o funcionalidad escrita desde la necesidad del usuario/cliente.	
3C	Significado
Card / Tarjeta	Parte visible de la historia, donde se escribe su sintaxis.
Conversation	La más importante: intercambio entre Product Owner y equipo para construir conocimiento conjunto.
Confirmation	Criterios de aceptación y pruebas de usuario necesarias para validar la historia.

Sintaxis

Forma de la Card
En tus notas: Como <nombre del rol>, yo puedo <actividad>, de forma tal que <valor que recibo>.
En el apunte complementario: Como [rol], quiero [acción], para [valor de negocio].
Debe especificarse el rol exacto; se remarca evitar el genérico “Como usuario”.
Debe dejar claro quién, qué y para qué/por qué.
Criterios de aceptación
La confirmación incluye criterios de aceptación y pruebas de usuario para demostrar al Product Owner que la funcionalidad satisface lo esperado.

9. Modelo INVEST

Letra	Condición	Interpretación
I	Independent	Independiente.
N	Negotiable	Negociable: fija el qué y permite conversar el cómo.
V	Valuable	Valiosa: tiene un para qué o valor de negocio claro.
E	Estimable	El equipo puede dimensionar su tamaño.
S	Small	Pequeña: entra en un Sprint/iteración y se completa.
T	Testable	Tiene criterios y pruebas que demuestran que funciona.

10. Spikes y restricciones / RNF

Spike
Si una historia tiene demasiada incertidumbre técnica o funcional y no puede estimarse, falla la E de INVEST y se convierte en un Spike.
En las notas aparecen Spikes funcionales y técnicos.
El apunte complementario lo describe como una historia de investigación con tiempo asignado para resolver dudas.
RNF / Tech Stories
Si un RNF aplica específicamente a una historia, puede incorporarse en sus criterios de aceptación.
Si es transversal a toda la aplicación, se registra como Tech Story / Restricción.
Estas restricciones transversales no tienen estimación/peso propio de valor funcional, pero deben contemplarse al estimar historias funcionales.

11. User Story completa

Elementos presentes en las fuentes
Expresión / sintaxis completa de la Card.
Frase verbal y rol concreto.
Valor o para qué.

Criterios de aceptación.
Pruebas de usuario.
Cumplimiento de INVEST para estar lista para ingresar a una iteración/Sprint.